

Selecting Polynomial Coefficients for Synthetic Regression and Curve-Fitting Data

Status

Recommended design.

This note describes how to generate random polynomial functions that will subsequently be used to create synthetic datasets for polynomial regression and curve fitting.

The important requirement is not merely that the generated polynomial should *look reasonable*. The generated data must also support a numerically well-behaved regression experiment after random x -values are selected and noise or jitter is added.

For that reason, the polynomial should not be generated or fitted primarily in the ordinary power basis

$$1, x, x^2, x^3, \dots$$

Instead, the recommended default is to use a **Legendre polynomial basis over a normalized domain**, while retaining the ability to convert the resulting polynomial into ordinary power-series coefficients for display.

1 The problem being solved

Suppose we want to generate synthetic data from a degree- n polynomial

$$y = a_0 + a_1x + a_2x^2 + \dots + a_nx^n.$$

A straightforward implementation might:

1. randomly choose the coefficients a_i ;
2. randomly choose x -values;
3. evaluate the polynomial;
4. add random noise;
5. fit a degree- n polynomial to the resulting observations.

There are two separate numerical problems with doing this directly in the power basis.

1.1 Higher powers can dominate the generated curve

If

$$x \in [-5, 5],$$

then at $x = 5$ the basis functions

$$1, x, x^2, x^3, \dots$$

have magnitudes

$$1, 5, 25, 125, \dots$$

Equal-sized coefficients therefore do not produce equal-sized effects on the generated curve.

An earlier proposed solution was to shrink the coefficient range approximately as

$$a_i \propto \frac{1}{x_{\text{scale}}^i}.$$

That solves the **curve magnitude** problem, but it does not solve the regression problem.

1.2 Polynomial regression uses a Vandermonde matrix

When the synthetic data is subsequently fitted in the power basis, the regression uses a design matrix of the form

$$V = \begin{pmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^n \\ 1 & x_2 & x_2^2 & \cdots & x_2^n \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_m & x_m^2 & \cdots & x_m^n \end{pmatrix}.$$

This is a Vandermonde matrix.

The important point is that **the coefficients a_i do not appear in this matrix**. Consequently, making a_n very small does not improve the conditioning of the regression problem.

The synthetic observations can be written as

$$\mathbf{y} = V\mathbf{a} + \boldsymbol{\epsilon},$$

where $\boldsymbol{\epsilon}$ is the added jitter. In ordinary least squares,

$$\hat{\mathbf{a}} = (V^T V)^{-1} V^T \mathbf{y} = \mathbf{a} + (V^T V)^{-1} V^T \boldsymbol{\epsilon}.$$

If V is badly conditioned, small perturbations in $\boldsymbol{\epsilon}$, or even differences in the randomly selected x_i , can produce disproportionately large changes in the estimated coefficients.

Therefore coefficient shrinking alone is not an adequate solution for this generator.

2 Normalize the domain

The first step should always be to map the requested x -domain

$$[x_{\min}, x_{\max}]$$

onto

$$[-1, 1].$$

Define the midpoint

$$m = \frac{x_{\min} + x_{\max}}{2}$$

and the half-width

$$h = \frac{x_{\max} - x_{\min}}{2}.$$

Then define the normalized coordinate

$$t = \frac{x - m}{h}.$$

Thus,

$$x = x_{\min} \Rightarrow t = -1, \quad x = x_{\max} \Rightarrow t = 1.$$

For example, if $x \in [-5, 5]$, then

$$t = \frac{x}{5}.$$

If instead $x \in [95, 105]$, then

$$t = \frac{x - 100}{5}.$$

In both cases the polynomial is evaluated internally over the same normalized interval $t \in [-1, 1]$.

NumPy's polynomial convenience classes support a domain-to-window mapping, so normal evaluation and fitting can use the original x values while the class handles the normalized internal coordinate.

3 Use Legendre polynomials as the default basis

Instead of generating

$$p(t) = a_0 + a_1t + a_2t^2 + \cdots + a_nt^n,$$

generate

$$p(t) = c_0P_0(t) + c_1P_1(t) + c_2P_2(t) + \cdots + c_nP_n(t)$$

where P_n is the n th Legendre polynomial.

The first few are

$$\begin{aligned} P_0(t) &= 1, \\ P_1(t) &= t, \\ P_2(t) &= \frac{1}{2}(3t^2 - 1), \\ P_3(t) &= \frac{1}{2}(5t^3 - 3t). \end{aligned}$$

They are still ordinary polynomials; we are simply using a different set of polynomial building blocks.

The significant property for this application is their orthogonality:

$$\int_{-1}^1 P_j(t)P_k(t) dt = 0 \quad \text{for } j \neq k.$$

Their weight function is constant, which makes Legendre polynomials particularly appropriate if the generator chooses normalized x values approximately uniformly over $[-1, 1]$.

If

$$t_i \sim U(-1, 1),$$

then, as the sample becomes representative of the interval, the sampled Legendre basis functions tend to be much less correlated with one another than the corresponding power-basis columns

$$1, t, t^2, t^3, \dots$$

This improves the numerical stability of estimating coefficients from noisy observations.

4 Why Legendre rather than Chebyshev?

Chebyshev polynomials remain an excellent choice, and they correctly address the central issue that an orthogonal polynomial basis is preferable to the raw monomial basis for higher-degree numerical work.

For this specific generator, however, Legendre polynomials are a slightly better default **if the synthetic x values are drawn uniformly from the requested interval**.

Legendre polynomials are orthogonal on $[-1, 1]$ with constant weight

$$w(t) = 1.$$

Chebyshev polynomials are orthogonal with weight

$$w(t) = \frac{1}{\sqrt{1-t^2}},$$

which gives greater weight to observations near the endpoints.

Therefore the default rule should be:

- use **Legendre** when x is sampled uniformly;
- consider **Chebyshev** if the sampling scheme is deliberately Chebyshev-like, or if Chebyshev-specific approximation properties are desired;
- do not use the raw power basis as the primary numerical representation merely because its coefficients are easier to display.

The presentation format and the numerical representation need not be the same.

5 Recommended NumPy representation

Use NumPy's class-based polynomial API:

```
from numpy.polynomial import Legendre
```

A polynomial with Legendre coefficients

```
coef = [c0, c1, c2, ..., cn]
```

over a requested domain can be represented as

```
p = Legendre(
    coef,
    domain=[x_min, x_max]
)
```

Conceptually, NumPy interprets this as

$$p(x) = c_0 P_0(t) + c_1 P_1(t) + \cdots + c_n P_n(t),$$

where t is the normalized coordinate corresponding to x .

The polynomial can then be evaluated directly at the original x values:

```
y_true = p(x)
```

No explicit normalization is required at the call site.

6 Generating the coefficients

The generator should distinguish between:

1. **shape coefficients**, which determine the curvature of the function;
2. the **constant coefficient**, which controls vertical displacement;
3. the overall **signal scale**, which determines the vertical variation of the generated curve.

A simple initial draw is

```
coef = rng.uniform(-1.0, 1.0, degree + 1)
```

but this should be regarded as generation of a random *shape*, not as the final vertical scaling of the polynomial.

6.1 Ensure that the requested degree is meaningful

If a degree- n polynomial is requested, a purely uniform draw could accidentally produce

$$c_n \approx 0,$$

making the generated curve effectively lower degree.

The highest-order coefficient should therefore not be allowed to be arbitrarily close to zero. For example, generate its magnitude separately from a configurable interval

$$|c_n| \in [c_{\min}, 1]$$

and randomly select its sign.

The exact value of $c_{\min}$ is a design parameter rather than a mathematical constant. Its purpose is simply to ensure that a requested degree- n polynomial genuinely contains an n th-degree component.

7 Normalize the generated signal rather than inventing degree-dependent coefficient rules

Even with a well-behaved basis, summing more basis functions can increase the overall vertical range of the generated polynomial.

Rather than inventing a rule such as

$$c_i \propto \frac{1}{5^i},$$

generate a random shape first and then scale the entire non-constant part of the polynomial to a desired signal magnitude.

A reasonable procedure is:

1. generate random coefficients;
2. set the constant coefficient temporarily to zero;
3. construct the Legendre polynomial;
4. evaluate it over a reasonably dense grid covering the domain;
5. measure its standard deviation;
6. multiply all non-constant coefficients by a common factor so that the polynomial has the requested signal standard deviation.

If the generated shape has

$$\sigma_p = \text{std}(p(x)),$$

and the desired signal scale is s , use

$$\alpha = \frac{s}{\sigma_p}$$

and replace

$$c_i \leftarrow \alpha c_i \quad i \geq 1.$$

This is preferable to scaling different coefficients by different arbitrary degree-dependent factors. Multiplying every non-constant coefficient by the same number changes only the overall vertical scale of the generated shape. It does not alter the relative mixture of its polynomial components.

The constant term can then be generated independently if a random vertical offset is desired.

8 Optional refinement: equal expected contribution from different orders

The ordinary Legendre polynomials satisfy

$$\int_{-1}^1 P_n(t)^2 dt = \frac{2}{2n+1}.$$

Consequently, if coefficients of every order are drawn from exactly the same distribution, higher-order Legendre components have somewhat smaller average squared contribution under uniform sampling.

If it later becomes desirable for each polynomial order to have approximately equal expected energy, an orthonormalized form can be used:

$$\phi_n(t) = \sqrt{2n+1} P_n(t).$$

Then, for $t \sim U(-1, 1)$,

$$E[\phi_n(t)^2] = 1.$$

In NumPy this does not require implementing another polynomial class. Generate latent coefficients β_n from a common distribution and store the corresponding standard Legendre coefficients as

$$c_n = \sqrt{2n+1} \beta_n.$$

This refinement is **not necessary for the first implementation**. Generating a random Legendre shape and normalizing its final signal magnitude is simpler and probably sufficient.

9 Generate synthetic observations

Once the noiseless polynomial $p(x)$ has been constructed, draw the independent-variable values from the requested domain:

```
x = rng.uniform(x_min, x_max, size=n_samples)
```

Evaluate

```
y_true = p(x)
```

and then add noise:

```
y = y_true + noise
```

For reproducible regression difficulty, the noise magnitude should preferably be defined relative to the generated signal rather than as an unrelated absolute number. For example,

$$\sigma_\epsilon = r \text{std}(y_{\text{true}}),$$

where r is a configurable noise ratio.

Gaussian noise could then be generated as

```
noise_std = noise_ratio * np.std(y_true)
noise = rng.normal(0.0, noise_std, size=n_samples)
```

This makes `noise_ratio` approximately describe how noisy the dataset is independently of the particular randomly generated polynomial.

10 Fit in the same basis used for generation

This is critical.

Generating the true polynomial in a Legendre basis but then fitting

$$1, x, x^2, \dots, x^n$$

does **not** solve the conditioning problem. The ill-conditioning belongs to the regression design matrix, not to the coefficients used to create y .

Therefore the primary regression should also use the Legendre basis:

```
fitted = Legendre.fit(
    x,
    y,
    deg=degree,
    domain=[x_min, x_max]
)
```

The generator and fitter should use the same domain so that their Legendre coefficients have the same interpretation.

Thus the experiment becomes

true Legendre coefficients $\longrightarrow$ noisy observations $\longrightarrow$ estimated Legendre coefficients

The fitted coefficients can then be compared directly with the coefficients used to generate the data.

11 Convert to ordinary polynomial coefficients for display

There will still be cases where the polynomial should be displayed in the familiar form

$$y = a_0 + a_1x + a_2x^2 + \dots + a_nx^n.$$

That does not require fitting in the power basis.

NumPy can convert the Legendre representation into an equivalent ordinary polynomial:

```
from numpy.polynomial import Polynomial

power_form = p.convert(kind=Polynomial)
power_coef = power_form.coef
```

Thus the implementation can maintain two representations:

Canonical numerical representation	Legendre coefficients
Presentation representation	Ordinary power coefficients

The latter should be treated mainly as a display or export format.

For example, a cubic can still be shown to the user as

$$y = ax^3 + bx^2 + cx + d$$

even though it was generated and fitted internally using Legendre polynomials.

12 Conditioning should be observable

Because the purpose of this component is to generate datasets for regression experiments, conditioning should not be treated as an invisible implementation detail.

For diagnostic or testing purposes, construct the Legendre Vandermonde matrix using the normalized coordinates:

```
from numpy.polynomial.legendre import legvander

m = (x_min + x_max) / 2
h = (x_max - x_min) / 2
t = (x - m) / h

V = legvander(t, degree)
condition_number = np.linalg.cond(V)
```

The generator does not necessarily need to reject every dataset above some fixed condition-number threshold. There is no universal threshold appropriate to every experiment.

However, recording the condition number is useful because random x selection can occasionally produce an unfortunate set of sample points even when a good basis is being used. A later version may choose to reject and resample unusually poorly conditioned point sets.

13 Sampling density still matters

Changing basis does not make every possible regression problem well-conditioned.

A degree- n polynomial has $n + 1$ coefficients, so at least $n + 1$ distinct observations are required even for an exact noiseless solution. With noise present, using only slightly more than $n + 1$ observations will generally make coefficient estimates much more sensitive to the particular random sample.

Synthetic datasets intended to demonstrate regression should therefore use comfortably more observations than coefficients unless the purpose of the experiment is specifically to demonstrate interpolation or underdetermination.

Similarly, an unfortunate random draw in which most x values cluster in a small portion of the domain can still produce a poor regression problem. Orthogonal polynomials improve conditioning; they do not eliminate the need for sensible sampling.

14 Recommended algorithm

For requested degree n , domain $[x_{\min}, x_{\max}]$, and number of observations N :

Polynomial generation

1. Generate $n + 1$ random Legendre coefficients.
2. Ensure the degree- n coefficient is not negligibly small.
3. Temporarily set $c_0 = 0$.
4. Construct `Legendre(coef, domain=[x_min, x_max])`.

5. Evaluate the polynomial over a dense grid.
6. Scale all c_i , $i \geq 1$, by a common factor so that the generated curve has the requested signal magnitude.
7. Generate c_0 separately if a vertical offset is desired.
8. Reconstruct the final **Legendre** instance.

Synthetic data generation

9. Draw

$$x_i \sim U(x_{\min}, x_{\max}).$$

10. Evaluate

$$y_i^* = p(x_i).$$

11. Choose noise magnitude relative to the signal:

$$\sigma_\epsilon = r \operatorname{std}(y^*).$$

12. Generate noisy observations:

$$y_i = y_i^* + \epsilon_i.$$

Regression

13. Fit using `Legendre.fit(x, y, degree, domain=[x_min, x_max])`.
14. Compare the true and fitted Legendre coefficient arrays.
15. Optionally convert either polynomial to `Polynomial` for familiar power-series display.
16. Optionally record the Legendre Vandermonde condition number and fitting diagnostics.

15 What not to do

Do not use degree-dependent coefficient shrinking as the main solution

A rule such as

$$a_i \propto \frac{1}{x_{\text{scale}}^i}$$

can keep a raw polynomial visually bounded, but it does not improve the conditioning of the Vandermonde matrix used later for regression.

Do not generate in Legendre and then fit in raw powers merely for convenience

That reintroduces the very conditioning problem that motivated the change of basis. Convert to power-series coefficients *after fitting* if they are needed for display.

Do not use `numpy.polyfit` as the default implementation

Prefer the modern `numpy.polynomial` class-based APIs for this design, particularly **Legendre** and `Legendre.fit`.

Do not assume an orthogonal basis guarantees a good random dataset

The number and distribution of sampled x values still matter.

16 Proposed API-level outputs

It may be useful for the dataset generator to retain both the canonical and presentation forms:

```
true_basis:
    "legendre"

true_coefficients:
    coefficients in the Legendre basis

true_domain:
    [x_min, x_max]

true_power_coefficients:
    optional coefficients in the ordinary  $x^n$  basis

x:
    sampled x values

y_true:
    noiseless generated values

y:
    noisy observations

noise_std:
    actual noise standard deviation

condition_number:
    optional condition number of the regression design matrix
```

The meaning of `true_coefficients` should always be accompanied by the basis. A coefficient array has no complete mathematical meaning without knowing which basis it belongs to.

17 Final recommendation

The previous coefficient-shrinking approach solved the wrong half of the problem.

It answered:

How can we prevent randomly generated high-degree polynomial terms from making the graph explode?

But the actual requirement is broader:

How can we generate a random polynomial, add noise, and subsequently recover its coefficients using a regression problem that is not unnecessarily sensitive to noise and random x selection?

For uniformly sampled x values, the recommended design is therefore

normalize domain $\rightarrow$ generate in Legendre basis $\rightarrow$ add noise $\rightarrow$ fit in Legendre basis

and, only when needed,

Legendre representation $\rightarrow$ ordinary power representation for display

.

This separates the numerical representation from the presentation representation and gives the synthetic regression experiment substantially better numerical foundations than generating and fitting directly in the raw monomial basis.

NumPy references

The implementation should rely primarily on the following NumPy APIs:

- `numpy.polynomial.Legendre`
- `numpy.polynomial.Legendre.fit`
- `numpy.polynomial.Polynomial`
- `numpy.polynomial.legendre.legvander`
- `numpy.linalg.cond`